

h1>

CSE 3206 Software Engineering Sessional Rajshahi University of Engineering & Technology (RUET), Rajshahi-6204, Bangladesh Department of Computer Science & Engineering Lab 2 — Software Process Models, Requirement Analysis & MVP Development Project #7 — Blood Donation Management System Agile Scrum 10 Heaven's light is our guide

— — —

* LifeFlow — Emergency Blood Donation Matching & Central Reserve Management System
* Department of Computer Science & Engineering, RUET * August 15, 2026

— — —

Team Member	Student Name	Roll Number	Role	Key Contributions	:--- :--- :--- :--
--	:---	`2203019`		Repository setup, React Context state engine	
				(`AppContext.jsx`), Authentication (`AuthModal.jsx`), Profile management	
				(`ProfileModal.jsx`), Role Security.	`2203020`
				Donor Search Directory	
				(`DonorDirectory.jsx`, `DonorCard.jsx`), Blood Group & Location Filtering, Emergency	
				Request Modal (`EmergencyRequestModal.jsx`), Request Tracker (`MyRequests.jsx`).	
				`2203021`	
				Analytics Dashboard (`MetricsOverview.jsx`), 8-Blood-Group Reserve Matrix	
				(`InventoryMatrix.jsx`), Admin Approval Queue (`AdminApprovalQueue.jsx`), Activity Audit	
				Feed (`ActivityLog.jsx`).	

— — —

is an emergency blood donation matching and central inventory management platform designed to connect voluntary blood donors with patients in urgent medical need across Bangladesh. Built as a single-page React application (MVP) with glassmorphism aesthetics and local state synchronization, LifeFlow eliminates critical time delays in locating compatible blood bags during medical emergencies (accidents, surgeries, dengue platelet emergencies, and thalassemia transfusions).

— — —

In Bangladesh, finding voluntary blood donors during medical emergencies remains a fragmented and high-friction process. Hospitals and blood banks often experience acute shortages of rare blood groups (e.g., `O-`, `B-`, `AB-`). Patients' relatives are forced to rely on frantic social media posts or personal contact networks, leading to dangerous delays in blood transfusion. Key pain points include:

1. : Relatives call donors who are unavailable, out of town, or have recently donated blood.
2. : Patients cannot easily view available blood bags in hospital reserve storage.
3. : Emergency requests lack urgency prioritization and donor targeting mechanisms.
4. : Publicly broadcasting personal phone numbers without access control exposes donors to spam.

1. : Enable patients/requesters to filter and contact eligible voluntary donors within minutes based on blood group and district. 2. : Provide a visual 8-blood-group inventory matrix showing reserve levels and low-stock alerts. 3. : Implement structured emergency blood request submission with urgency classification (🔥, ⚡, 📄). 4. : Protect sensitive donor management and request fulfillment actions through distinct roles (, , and). 5. : Follow iterative sprint planning, component modularity, and collaborative Git workflow.

Stakeholder Group	Primary Goals & Needs	System Interaction Level
Manage availability status (`Available` / `Unavailable`), track donation history, receive direct donor requests.	Authenticated User (`Donor` Role)	Search compatible donors by blood group/location, submit emergency requests, track request status.
Authenticated / Guest (`Recipient` Role)	Monitor reserve bag stock levels, adjust inventory counts, coordinate emergency blood allocation.	Authenticated User (`Admin` Role)
Authenticate User (`Admin` Role)	Approve pending emergency requests, oversee activity logs, manage platform security.	Authenticated User (`Admin` Role)

> [!NOTE] >

* : The system shall allow users to register with Full Name, Email, Password, Phone Number, Blood Group, District, and Primary Role (`Donor`, `Recipient`, `Admin`). * : The system shall validate login credentials and persist active session state in local storage. * : The

system shall dynamically adjust navigation and control options based on user role (Admin controls vs Donor availability controls). * : The system shall display verified donor cards with blood group badges, district, availability status, and last donation date. * : The system shall allow real-time filtering of donors by Blood Group (`A+`, `A-`, `B+`, `B-`, `O+`, `O-`, `AB+`, `AB-`), District, and Availability status (`Available Only`). * : The system shall provide an Emergency Request Modal collecting Patient Name, Required Blood Group, Units Needed, Hospital Name, District, Urgency Level, and Medical Reason. * : The system shall allow requesters to submit direct emergency requests targeted to a specific voluntary donor. * : The system shall track request statuses (`Pending`, `Approved`, `Fulfilled`, `Cancelled`, `Rejected`) in real time. * : The system shall allow users to toggle between viewing their own personal requests and viewing system-wide requests across Bangladesh. * : The system shall restrict request cancellation and fulfillment controls strictly to the request creator or an Admin (`View Only` mode for unauthorized users). * : The system shall render a live 8-blood-group reserve matrix with unit counts, progress bars, and status indicators (`Sufficient`, `Low Stock`, `Critical`). * : The system shall allow Admin users to approve pending blood requests, allocate blood units, and adjust inventory counts (`+` / `-`).

> [!NOTE] >

* : The application shall load initial UI assets in under and process local state filters in under . * : The UI shall use high-contrast glassmorphism themes (`#0B0F19` dark background, `#E11D48` crimson accents) and legible Google Fonts (`Plus Jakarta Sans` & `Inter`). * : The application layout shall adapt seamlessly across screen widths from (mobile) up to (ultra-wide desktop) without text clipping or horizontal scroll overflow. * : The system shall synchronize all user registrations, donor availability updates, blood requests, and inventory counts to browser `localStorage`. * : Unauthenticated and non-admin users shall be prohibited from executing admin actions or modifying third-party requests. * : The system shall handle missing or null data fields gracefully without throwing unhandled JavaScript runtime exceptions. * : Code shall follow strict React component isolation (`Auth/`, `Donor/`, `Dashboard/`, `Common/`) to support semester-long lab extensions. * : Badges, buttons, and status indicators shall enforce `whitespace-nowrap` and flex alignment to prevent icon clipping or line breaks. * : The central state architecture (`AppContext.jsx`) shall support scaling up to 1,000+ donor records in memory. * : The client app shall execute on any modern web browser (Chrome, Firefox, Edge, Safari) without requiring external server setup.

* distressed relative of a patient, * filter available voluntary donors by blood group `O+` and district `Rajshahi`, * I can instantly find eligible donors near the hospital. * : * Given the user is on the "Find Donors" tab. * When the user selects `O+` and district `Rajshahi`. * Then only available `O+` donors in Rajshahi are displayed within 50ms.

* registered voluntary blood donor, * toggle my status between `Available` and `Unavailable` from my profile drawer, * requesters only contact me when I am eligible to donate. * : * Given the donor is logged in. * When the donor clicks "Toggle Status" in the profile modal. * Then their availability updates in real time across the donor directory.

* hospital requester, * fill out an Emergency Blood Request form with urgency level `Critical`, * available donors and blood bank admins are alerted immediately. * : * Given the user opens the Emergency Request Modal. * When all required fields (patient name, hospital, units, blood group) are submitted. * Then a success toast notification appears and the request enters `Pending` state.

* requester who submitted a blood request, * mark my request as `Fulfilled` once blood is received, * donors know the request is completed. * : * Given a logged-in user views "My Personal Requests". * When the user clicks "Mark Fulfilled" on their active request. * Then the status badge updates to `Fulfilled` and action buttons are disabled. * Unauthenticated users or non-owners see `View Only` mode for other users' requests.

* Blood Bank Administrator, * review pending emergency requests and approve unit allocations from reserve storage, * hospital inventory is officially allocated. * : * Given the user is logged in as `admin@lifeflow.org`. * When the admin clicks "Approve & Allocate" in the Admin Approval Queue. * Then the request status becomes `Approved` and the respective blood group inventory count automatically deducts 1 bag.

* blood bank manager, * view visual progress bars and low-stock alerts for all 8 blood groups, * I can organize urgent donor drives when stock falls below 5 units. * : * Given the

user views the "Blood Inventory" tab. * When any blood group reserve falls below 3 units. * Then the status badge automatically displays `🔥 Critical` with a red highlight.

For , was selected as the optimal software process model.

```
```mermaid graph TD
 A["Sprint Planning (Requirements & Roles)"] --> B["Sprint 1: Core Store & Auth Module"]
 B --> C["Sprint 2: Donor Search & Emergency Requests"]
 C --> D["Sprint 3: Dashboard, Inventory & Admin Queue"]
 D --> E["Sprint Review & MVP Demonstration"]
  ```
```

* * Setup Vite React project structure, Tailwind CSS glassmorphic theme, and Google Fonts. * Build central state context (`AppContext.jsx`) with pre-seeded donor and inventory data. * Implement `AuthModal.jsx` (Login/Register) and `ProfileModal.jsx` with donor availability toggle. * * Implement `DonorDirectory.jsx` and `DonorCard.jsx` with multi-criteria filtering. * Implement `EmergencyRequestModal.jsx` and `MyRequests.jsx` with access-controlled actions. * * Implement `MetricsOverview.jsx` KPI summary cards and `InventoryMatrix.jsx` (8 blood groups). * Implement `AdminApprovalQueue.jsx` and `ActivityLog.jsx` real-time audit feed.

1. : Software engineering labs evolve across the semester. Scrum's sprint-based architecture allows incorporating future lab requirements (design patterns, unit testing, CI/CD) without redesigning the core. 2. : Scrum prioritizes delivering a working prototype early. By prioritizing core modules (Auth, Donor Search, Inventory), the team produced a functional MVP quickly. 3. : Scrum user stories map cleanly to component ownership. Member 1 (Auth), Member 2 (Donor Search), and Member 3 (Dashboard) worked in parallel without blocking each other. 4. : Short iteration cycles allowed rapid testing and refinement of UI spacing, access control security, and responsiveness.

| | | | |
|---|----------------------------------|-----------------------|------|
| Evaluation Criteria Agile Scrum (Selected) Waterfall Model Spiral Model Rapid Application Dev (RAD) | :--- :--- :--- :--- :--- | Low Moderate High | Slow |
|---|----------------------------------|-----------------------|------|

(Sequential) | Moderate | Fast | | | Late (Testing Phase) | High (Risk Analysis) | Moderate | | |
 | Unsuitable | Overkill for MVP | Moderately Suitable |

* : Waterfall requires fixed, frozen requirements up front. If requirement changes occur during semester-long lab milestones, Waterfall forces expensive backtracking. Testing happens only at the end. * : The Spiral model emphasizes extensive risk analysis and formal phase gates in large enterprise/safety-critical systems. For a small 3-member lab MVP, the overhead of formal risk analysis phases would delay core feature development.

```

``mermaid graph TD
  App["App.jsx (Main Layout)"] --> Navbar["Navbar.jsx (Header & Tabs)"]
  App --> AppContext["AppContext.jsx (State Engine)"]
  App --> MainContent["MainContent Router"]
  MainContent --> DashboardView["Dashboard View"]
  MainContent --> DonorView["Donor Directory View"]
  MainContent --> InventoryView["Blood Inventory View"]
  MainContent --> RequestsView["My Requests View"]
  DashboardView --> MetricsOverview["MetricsOverview.jsx"]
  DashboardView --> InventoryMatrix["InventoryMatrix.jsx"]
  DashboardView --> AdminApprovalQueue["AdminApprovalQueue.jsx"]
  DashboardView --> ActivityLog["ActivityLog.jsx"]
  
```

```

DonorView --> DonorDirectory["DonorDirectory.jsx"]
DonorDirectory --> DonorCard["DonorCard.jsx"]
  
```

```

RequestsView --> MyRequests["MyRequests.jsx"]
  
```

```

App --> AuthModal["AuthModal.jsx"]
App --> ProfileModal["ProfileModal.jsx"]
App --> EmergencyRequestModal["EmergencyRequestModal.jsx"]
App --> Toast["Toast.jsx"]
``
  
```

The team strictly enforced the branching and Pull Request strategy outlined in [lab_2.md] (file:///Users/apon/3-2/CSE%203206/Lab-2/lab_2.md#L104-L124):


```

` ``text Blood_Donation_Mangement/ |—— README.md # Full setup, team task allocation
& Git workflow |—— docs/ | —— Requirement Report.pdf # Project Design Report
|—— screenshots/ # Working MVP screenshots |—— package.json |—— vite.config.js
|—— src/ |—— index.css |—— main.jsx |—— App.jsx |—— context/ | ——
AppContext.jsx |—— components/ |—— Common/ |—— Auth/ |—— Donor/ ——
Dashboard/ ` ``

```

1. `main`: Protected production branch. 2. `feature/auth-user-management` (Member 1): App context, auth modal, profile drawer, navbar layout. 3. `feature/donor-search-requests` (Member 2): Donor directory, filters, emergency request modal, request table. 4. `feature/dashboard-inventory` (Member 3): Metrics overview cards, 8-blood-group inventory matrix, admin approval panel, audit feed.

1. Symptom*: Table action buttons wrapped awkwardly and status badge icons clipped adjacent borders on medium screens. Mitigation*: Expanded container width from `max-w-7xl` to `max-w-[1536px]`, added `whitespace-nowrap` to badges/buttons, and replaced raw inline emojis with Lucide icons (`Flame`, `Zap`, `CheckCircle`) with `shrink-0`. 2. Symptom*: Unauthenticated guests could click "Mark Fulfilled" or "Cancel" on system-wide emergency requests. Mitigation*: Implemented strict `canManageRequest(req, currentUser)` permission check in [MyRequests.jsx](file:///Users/apon/3-2/CSE%203206/Lab-2/Blood_Donation_Mangement/src/components/Donor/MyRequests.jsx), restricting actions to the request owner or an Admin (`View Only` mode for unauthorized users). 3. Symptom*: The internal Admin Approval Queue box was visible to regular donors and guests. Mitigation*: Updated [AdminApprovalQueue.jsx](file:///Users/apon/3-2/CSE%203206/Lab-2/Blood_Donation_Mangement/src/components/Dashboard/AdminApprovalQueue.jsx) to enforce `if (!isAdmin || pendingRequests.length === 0) return null;`, ensuring administrative controls only appear for `admin@lifeflow.org`.

The successfully fulfills all objectives for . By combining thorough requirement analysis, an Agile Scrum development methodology, a responsive React component architecture, and a

structured GitHub collaboration workflow, the team delivered a life-saving prototype ready for semester-long expansion.

* : Apply Singleton Pattern for `AppContext` and Factory/Observer Patterns for real-time notification dispatching. * : Implement Unit Tests (Jest/React Testing Library) for state reducers and E2E Tests (Playwright/Cypress) for request submission flows. * : Configure GitHub Actions CI pipeline and deploy live production build to Vercel/Netlify.